

RODRIGO ARAÚJO VIANNINI

POSTECH

IA PARA DEVS

COMPUTER VISION

AULA 04

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	22
MERCADO, CASES E TENDÊNCIAS	27
O QUE VOCÊ VIU NESTA AULA?	29
REFERÊNCIAS.....	30

EMSE

O QUE VEM POR AÍ?

Fala Devs! Nesta aula, vamos estudar as Redes Neurais Convolucionais (CNNs), fundamentais para a área de visão computacional. Começaremos entendendo a estrutura básica de uma CNN, suas camadas convolucionais, pooling e fully connected, e como esses componentes trabalham juntos para extrair e aprender características de imagens.

Exploraremos a evolução das arquiteturas de CNNs, desde os modelos pioneiros como LeNet-5 e AlexNet, até as redes mais avançadas como VGG, ResNet e EfficientNet. Analisaremos como cada inovação arquitetônica contribuiu para melhorias em desempenho e precisão, bem como seus impactos em benchmarks de visão computacional.

Em seguida, abordaremos a técnica de Transfer Learning aplicada a CNNs. Veremos como utilizar redes pré-treinadas para resolver problemas específicos, economizando tempo e recursos computacionais. Discutiremos as vantagens de ajustar modelos pré-treinados para novas tarefas, e como essa abordagem pode ser aplicada em diversos contextos de visão computacional.

Outro tópico essencial é a aplicação prática de CNNs na segmentação de imagens, utilizando redes como U-Net e Mask R-CNN. Estudaremos suas arquiteturas e como elas são utilizadas para tarefas de segmentação semântica e instância, essenciais para aplicações que requerem uma análise detalhada e precisa de cada pixel da imagem.

Por fim, vamos treinar e implementar modelos de CNN, para proporcionar a compreensão necessária das CNNs, de suas aplicações e melhores práticas, preparando você para enfrentar desafios avançados em visão computacional e desenvolver soluções robustas e eficientes.

HANDS ON

As Redes Neurais Convolucionais (CNNs) são um tipo de arquitetura de rede neural profundamente conectada, amplamente utilizada em problemas de visão computacional, como classificação de imagens, detecção de objetos e segmentação de imagens. Sua eficácia e popularidade resultam de sua capacidade de aprender automaticamente padrões complexos e hierárquicos diretamente dos dados brutos de entrada, sem a necessidade de extração manual de características.

ESTRUTURA BÁSICA DE UMA CNN

Uma CNN é composta por várias camadas, cada uma desempenhando um papel crucial no processo de aprendizado e extração de características. A estrutura básica de uma CNN é composta por camadas convolucionais, camadas de pooling e camadas totalmente conectadas.

CAMADAS CONVOLUCIONAIS

As camadas convolucionais são o componente fundamental das CNNs e são responsáveis por extrair características importantes das imagens de entrada. Cada camada convolucional consiste em um conjunto de filtros ou kernels, que são convolvidos com a imagem de entrada para produzir mapas de características. Esses filtros capturam padrões locais em diferentes partes da imagem, como bordas, texturas e formas. Durante o treinamento, os pesos dos filtros são ajustados automaticamente pela retropropagação do gradiente, permitindo que a rede aprenda a representação mais discriminativa dos dados.

As camadas convolucionais são o componente central das CNNs. Elas operam aplicando filtros (kernels) à entrada, executando uma operação de convolução para produzir mapas de características (feature maps). Cada filtro é treinado para detectar diferentes características locais na imagem, como bordas, texturas e padrões de cores.

FUNCIONAMENTO DAS CAMADAS CONVOLUCIONAIS

Um filtro convolucional é uma pequena matriz de pesos que desliza sobre a entrada (imagem), realizando a convolução. O processo pode ser descrito matematicamente pela fórmula:

$$(I * K)(x, y) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} I(x + i, y + j) \cdot K(i, j)$$

onde I é a imagem de entrada, K é o filtro, e (x, y) são as coordenadas do pixel no mapa de características resultante.

Figura 1 - Fórmula Filtro Convolucional
Fonte: Elaborado pelo autor (2024)

CARACTERÍSTICAS

- **Compartilhamento de pesos:** um mesmo conjunto de filtros é aplicado em toda a entrada, reduzindo a complexidade computacional.
- **Receptive field:** cada neurônio em um mapa de características convolucional é conectado a uma pequena região da entrada, conhecida como campo receptivo, o que permite a detecção de padrões locais.

CAMADAS DE POOLING

As camadas de pooling são intercaladas entre as camadas convolucionais e têm a função de reduzir a dimensionalidade dos mapas de características, preservando as características mais importantes. O pooling é realizado aplicando uma função (como max ou média) a uma região do mapa de características. Isso ajuda a tornar a representação mais robusta a pequenas variações na posição dos objetos na imagem e reduz o custo computacional da rede.

TIPOS DE POOLING

- **Max pooling:** seleciona o valor máximo em cada região da matriz de entrada.

- **Average pooling:** calcula a média dos valores em cada região da matriz de entrada.

FUNCIONAMENTO DAS CAMADAS DE POOLING

O pooling é realizado aplicando uma janela deslizante sobre o mapa de características e, em cada posição, computando o valor máximo ou médio dos elementos dentro da janela. Por exemplo, em uma janela 2x2, o max pooling seleciona o maior valor dentre os quatro elementos cobertos pela janela.

CAMADAS CONECTADAS (OU CAMADAS DENSAS)

As camadas totalmente conectadas, também conhecidas como camadas densas, estão localizadas no final da rede e são responsáveis por realizar a classificação com base nas características extraídas pelas camadas convolucionais e de pooling. Cada neurônio em uma camada totalmente conectada está ligado a todos os neurônios da camada anterior, permitindo que a rede aprenda combinações mais complexas de características.

CAMADAS FULLY CONNECTED

As camadas totalmente conectadas (fully connected) são semelhantes às camadas nas redes neurais tradicionais, onde cada neurônio é conectado a todos os neurônios da camada anterior.

FUNÇÃO DAS CAMADAS FULLY CONNECTED

- **Integração das características:** elas integram as características extraídas pelas camadas anteriores e realizam a classificação final ou a regressão.
- **Ativações:** utilizam funções de ativação como ReLU (Rectified Linear Unit) ou softmax, dependendo da tarefa específica (classificação ou regressão).

EVOLUÇÃO DAS ARQUITETURAS DE CNNs

A trajetória da evolução das Redes Neurais Convolucionais (CNNs) reflete uma jornada marcada pela constante inovação e refinamento, desde os modelos pioneiros até as arquiteturas avançadas que moldam o cenário atual da visão computacional.

MODELOS PIONEIROS: LENET-5, ALEXNET

- **LeNet-5:** desenvolvida por Yann LeCun e sua equipe em 1998, a LeNet-5 é um marco na história das CNNs, especialmente no reconhecimento de caracteres manuscritos. Composta por camadas convolucionais, de subamostragem (pooling) e totalmente conectadas, a LeNet-5 estabeleceu os fundamentos das redes convolucionais, utilizando convoluções 2D e compartilhamento de pesos.
- **AlexNet:** em 2012, a AlexNet, projetada por Alex Krizhevsky e colaboradores, revolucionou o campo da visão computacional ao vencer o desafio ImageNet com uma margem significativa. Além de múltiplas camadas convolucionais, a AlexNet introduziu o uso eficiente de GPUs para treinamento em larga escala e empregou funções de ativação não lineares como ReLU, assim como técnicas de regularização como dropout.

ARQUITETURAS AVANÇADAS: VGG, RESNET, EFFICIENTNET

- **VGG:** proposta em 2014 por Simonyan e Zisserman, a VGG adotou uma abordagem de profundidade crescente e convoluções 3x3. A uniformidade estrutural das arquiteturas VGG16 e VGG19 simplificou a análise e destacou a importância da profundidade na extração de características.
- **ResNet:** apresentada por Kaiming He et al. em 2015, a ResNet introduziu o conceito de conexões residuais, permitindo o treinamento de redes mais profundas e mitigando problemas de derretimento de gradiente. Essa inovação teve um impacto significativo na escalabilidade e generalização das CNNs.
- **EfficientNet:** desenvolvida em 2019 por Tan e Le, a EfficientNet adotou uma abordagem de escalonamento composto, otimizando simultaneamente

largura, profundidade e resolução da rede. Isso resultou em modelos altamente eficientes, com desempenho superior em relação à complexidade do modelo, sendo especialmente adequados para aplicações em dispositivos com recursos limitados.

IMPACTO EM BENCHMARKS DE VISÃO COMPUTACIONAL

Os avanços nas arquiteturas de CNNs têm um impacto direto nos benchmarks de visão computacional, como o desafio ImageNet. Métricas como precisão e eficiência são constantemente aprimoradas, refletindo a evolução contínua das CNNs. Modelos como EfficientNet estabelecem novos padrões de desempenho, impulsionando a pesquisa e o desenvolvimento de aplicações em diversos domínios.

TRANSFER LEARNING EM CNNs

O Transfer Learning é uma técnica poderosa em aprendizado profundo, particularmente aplicável a Redes Neurais Convolucionais (CNNs), que permite aproveitar o conhecimento adquirido em uma tarefa para resolver outra, geralmente mais específica ou relacionada. Esta abordagem envolve reutilizar modelos pré-treinados em grandes conjuntos de dados para tarefas semelhantes, ao invés de treinar um modelo do zero.

CONCEITO DE TRANSFER LEARNING

No transfer learning, um modelo pré-treinado em uma tarefa de alto volume de dados, como classificação de imagens em um dataset massivo como o ImageNet, é adaptado (transferido) para uma nova tarefa com dados de menor volume. Em vez de descartar todo o conhecimento aprendido pelo modelo durante o treinamento inicial, a ideia é aproveitar as representações de características generalizáveis aprendidas pelas camadas convolucionais para a nova tarefa.

UTILIZAÇÃO DE REDES PRÉ-TREINADAS

A utilização de redes pré-treinadas é uma das principais aplicações do Transfer Learning. Modelos populares como VGG, ResNet e EfficientNet, pré-treinados em grandes conjuntos de dados, podem ser baixados e usados como ponto de partida para diversas tarefas de visão computacional. Esses modelos capturam uma ampla variedade de características visuais, como bordas, texturas e formas, que podem ser relevantes para muitas outras tarefas.

VANTAGENS E APLICAÇÕES PRÁTICAS

As vantagens do transfer learning são diversas:

- Redução do tempo de treinamento: como o modelo já foi treinado em dados massivos, o treinamento para a nova tarefa é mais rápido e requer menos amostras.
- Melhor generalização: o modelo pré-treinado já aprendeu características genéricas úteis em imagens, o que pode ajudar na generalização para a nova tarefa, mesmo com menos dados.
- Necessidade de menos dados: como o modelo já possui uma boa compreensão das características das imagens, menos dados de treinamento são necessários para alcançar um desempenho satisfatório na nova tarefa.

O ajuste fino de modelos pré-treinados é comumente aplicado em uma variedade de tarefas, como:

- Classificação de imagens: adaptar um modelo pré-treinado para reconhecer classes específicas de objetos em imagens.
- Detecção de objetos: utilizar um modelo pré-treinado para identificar e localizar objetos específicos em uma imagem.
- Segmentação de imagens: ajustar um modelo pré-treinado para segmentar imagens em regiões de interesse, como partes do corpo humano ou objetos de interesse.

SEGMENTAÇÃO DE IMAGENS COM CNNs

A segmentação de imagens é uma tarefa crucial em visão computacional que envolve a atribuição de rótulos a cada pixel em uma imagem, identificando regiões de interesse e separando-as em diferentes categorias. O uso de Redes Neurais Convolucionais (CNNs) revolucionou a segmentação de imagens, permitindo abordagens mais eficazes e precisas.

SEGMENTAÇÃO SEMÂNTICA

Na segmentação semântica, o objetivo é atribuir um rótulo a cada pixel da imagem, indicando a classe à qual ele pertence. Isso significa que diferentes objetos na imagem são rotulados com cores distintas, sem distinguir instâncias individuais do mesmo objeto. A segmentação semântica é amplamente utilizada em aplicações como reconhecimento de objetos, navegação autônoma e monitoramento ambiental.

SEGMENTAÇÃO POR INSTÂNCIA

A segmentação por instância vai além da segmentação semântica, não apenas atribuindo um rótulo a cada pixel, mas também diferenciando entre instâncias individuais do mesmo objeto. Isso significa que cada objeto na imagem é identificado e segmentado separadamente, permitindo uma compreensão mais detalhada da cena. A segmentação por instância é crucial em aplicações como detecção de objetos, rastreamento de objetos e análise de vídeo.

ARQUITETURAS DE SEGMENTAÇÃO: U-NET, MASK R-CNN

- U-Net: a U-Net é uma arquitetura de CNN popularmente utilizada para tarefas de segmentação de imagens biomédicas, como segmentação de células e órgãos em imagens médicas. A U-Net possui uma estrutura de "encoder-decoder", onde as características são progressivamente reduzidas (encoder) e, em seguida, reconstruídas em uma resolução original (decoder), permitindo uma segmentação precisa e detalhada.

- **Mask R-CNN:** o Mask R-CNN é uma extensão do modelo Faster R-CNN, projetado para realizar detecção de objetos e segmentação de instâncias em uma única etapa. Ele combina uma rede neural convolucional para detecção de objetos com uma rede de máscara para segmentação de instâncias, permitindo a localização e segmentação precisa de múltiplos objetos na imagem.

TREINAMENTO E IMPLEMENTAÇÃO DE CNNs

O treinamento e implementação eficazes de Redes Neurais Convolucionais (CNNs) são fundamentais para alcançar um desempenho robusto em tarefas de visão computacional. Isso envolve uma série de etapas, desde a preparação dos conjuntos de dados até a otimização dos hiperparâmetros.

PREPARAÇÃO DE CONJUNTOS DE DADOS

A preparação adequada dos conjuntos de dados é essencial para o sucesso do treinamento de CNNs. Isso inclui:

- **Coleta de dados:** obtenção de conjuntos de imagens representativos e relevantes para a tarefa em questão.
- **Limpeza de dados:** remoção de imagens de baixa qualidade, duplicatas ou imagens irrelevantes que possam prejudicar o desempenho do modelo.
- **Anotação de dados:** rotulação precisa das imagens com as classes de interesse, garantindo que o modelo possa aprender a distinguir entre diferentes categorias.

TÉCNICAS DE DATA AUGMENTATION (AUMENTO DOS DADOS)

As técnicas de aumento de dados são utilizadas para aumentar a diversidade do conjunto de dados, tornando o modelo mais robusto e generalizável. Algumas técnicas comuns incluem:

- **Rotação:** girar a imagem em diferentes ângulos para simular diferentes perspectivas.

- Espelhamento: refletir horizontalmente ou verticalmente a imagem para criar variações espaciais.
- Corte e redimensionamento: cortar a imagem em diferentes regiões e redimensioná-la para diferentes tamanhos.
- Alteração de brilho e contraste: ajustar o brilho, contraste e saturação da imagem para simular diferentes condições de iluminação.

Essas técnicas ajudam a aumentar a quantidade de dados disponíveis para treinamento, reduzindo o risco de overfitting e melhorando a capacidade do modelo de generalizar para novos exemplos.

ESTRATÉGIAS DE REGULARIZAÇÃO PARA EVITAR OVERFITTING

O overfitting é um problema comum em modelos de aprendizado profundo, onde o modelo se adapta demais aos dados de treinamento e não generaliza bem para dados não vistos. Para evitar o overfitting, várias estratégias de regularização podem ser empregadas, incluindo:

- Dropout: desligar aleatoriamente um percentual dos neurônios durante o treinamento para evitar que o modelo se torne excessivamente dependente de determinadas características.
- Regularização L1 e L2: adicionar termos de penalização à função de perda para limitar os valores dos pesos do modelo, evitando que se tornem muito grandes.
- Batch normalization: normalizar as ativações das camadas intermediárias durante o treinamento para reduzir a covariância interna, ajudando na estabilidade do treinamento e prevenindo o overfitting.

OTIMIZAÇÃO DE HIPERPARÂMETROS

A otimização dos hiperparâmetros é uma etapa crítica no treinamento de CNNs, envolvendo a seleção dos valores ideais para parâmetros como taxa de aprendizado, tamanho do lote (batch size), número de camadas e tamanho dos filtros. Métodos como busca em grade, busca aleatória e otimização bayesiana são comumente

utilizados para encontrar os melhores hiperparâmetros para um determinado conjunto de dados e tarefa.

CNNS EM PRODUÇÃO

Para colocar Redes Neurais Convolucionais (CNNs) em produção, é necessário considerar várias etapas, desde a exportação dos modelos treinados até a otimização para inferência em dispositivos de edge computing.

EXPORTAÇÃO DE MODELOS TREINADOS

Após o treinamento de uma CNN, o próximo passo é exportar o modelo treinado para que ele possa ser implantado em produção. Isso geralmente envolve a conversão do modelo para um formato compatível com a biblioteca de inferência a ser utilizada, como TensorFlow Lite, ONNX ou TensorFlow.js, dependendo da plataforma de implantação.

INTEGRAÇÃO COM APLICAÇÕES REAIS

A integração de CNNs com aplicações reais pode ser realizada de várias maneiras, dependendo dos requisitos específicos do sistema. Algumas opções incluem:

- Desenvolvimento de APIs: criar uma API para a CNN que permite a integração com outras aplicações por meio de solicitações HTTP. Isso é útil quando a CNN precisa ser acessada remotamente por diferentes serviços ou aplicativos.
- Incorporação em aplicativos existentes: integrar a CNN diretamente em um aplicativo existente, seja como parte do código-fonte do aplicativo ou como uma biblioteca externa. Isso é comum em aplicativos móveis e de desktop que exigem funcionalidades de visão computacional.
- Implantação em servidores ou nuvem: hospedar a CNN em um servidor ou na nuvem, onde ela pode ser acessada por outros sistemas por meio de

chamadas de API. Isso é útil para aplicações web que exigem análise de imagens em tempo real.

OTIMIZAÇÃO PARA INFERÊNCIA EM DISPOSITIVOS DE EDGE COMPUTING

Para executar CNNs em dispositivos de edge computing, como smartphones, dispositivos IoT e sistemas embarcados, é importante otimizar os modelos para inferência eficiente em recursos limitados. Isso pode envolver:

- Quantização de pesos: reduzir a precisão dos pesos do modelo para inteiros de 8 bits para reduzir o tamanho do modelo e acelerar a inferência.
- Pods de inferência: utilizar aceleradores de hardware dedicados, como unidades de processamento tensorial (TPUs) ou unidades de processamento gráfico (GPUs), para acelerar a inferência de modelos de CNN em dispositivos de borda.
- Pruning (Poda): identificar e remover conexões redundantes ou pouco importantes no modelo para reduzir o tamanho do modelo e melhorar a eficiência computacional.
- Quantização dinâmica: ajustar dinamicamente a precisão dos pesos do modelo durante a inferência com base na distribuição dos dados de entrada.

Ao otimizar CNNs para inferência em dispositivos de edge computing, é possível garantir um desempenho eficiente e responsivo, mesmo em dispositivos com recursos limitados de computação e energia. Isso é crucial para aplicações que requerem análise de imagens em tempo real, como reconhecimento de objetos, detecção de rostos e monitoramento de segurança.

ESTRUTURA BÁSICA DE UMA CNN

As Redes Neurais Convolucionais (CNNs) são um tipo de rede neural profunda, amplamente utilizada para processamento e análise de imagens. A estrutura básica de uma CNN consiste em três tipos principais de camadas: camadas convolucionais, camadas de pooling e camadas fully connected. Vamos explorar cada uma dessas camadas em detalhes e, em seguida, implementar um exemplo prático.

CAMADAS CONVOLUCIONAIS

As camadas convolucionais são a espinha dorsal das CNNs. Elas aplicam convoluções a uma entrada, passando um filtro (ou kernel) sobre a entrada para produzir um mapa de características.

- Filtro (Kernel): uma pequena matriz de pesos, usada para extrair características locais da entrada.
- Stride: o número de pixels que o filtro se move a cada passo.
- Padding: adiciona zeros nas bordas da entrada para manter a dimensionalidade.

Vamos codar...

Preparando o ambiente:

```
1 !pip install tensorflow
2 !pip install keras=
3 !pip install tensorflow-datasets
4
```

Figura 2 – Código (1)

Fonte: Elaborado pelo autor (2024)

Importando as bibliotecas:

```
1 # Importando as bibliotecas
2 import numpy as np
3 import tensorflow as tf
4 from tensorflow.keras import datasets, layers, models
5 import matplotlib.pyplot as plt
```

Figura 3 – Código (2)

Fonte: Elaborado pelo autor (2024)

Exemplo de cálculo de convolução:

Suponha uma entrada 5x5 e um filtro 3x3:

```

1  import numpy as np
2
3  # Entrada 5x5
4  input_matrix = np.array([[1, 2, 3, 0, 1],
5                           [4, 5, 6, 1, 2],
6                           [7, 8, 9, 0, 1],
7                           [0, 1, 2, 3, 4],
8                           [1, 2, 3, 4, 5]])
9
10 # Filtro 3x3
11 filter_matrix = np.array([[1, 0, -1],
12                            [1, 0, -1],
13                            [1, 0, -1]])
14
15 # Realizando a convolução
16 output_matrix = np.zeros((3, 3))
17 for i in range(3):
18     for j in range(3):
19         region = input_matrix[i:i+3, j:j+3]
20         output_matrix[i, j] = np.sum(region * filter_matrix)
21
22 print(output_matrix)
23

```

Figura 4 – Código (3)
Fonte: Elaborado pelo autor (2024)

Output:

```

[[ -6.  14.  14.]
 [ -6.  10.  10.]
 [ -6.   4.   4.]]

```

Figura 5 – Código (4)
Fonte: Elaborado pelo autor (2024)

CAMADAS DE POOLING

As camadas de pooling são usadas para reduzir a dimensionalidade das características extraídas pelas camadas convolucionais. A forma mais comum é o max pooling, que pega o valor máximo de uma região.

Exemplo de max pooling:

```

1  # Entrada 4x4
2  input_matrix = np.array([[1, 3, 2, 4],
3                           [5, 6, 1, 2],
4                           [3, 0, 2, 1],
5                           [1, 2, 3, 0]])
6
7  # Aplicando max pooling 2x2
8  output_matrix = np.zeros((2, 2))
9  for i in range(2):
10     for j in range(2):
11         region = input_matrix[i*2:i*2+2, j*2:j*2+2]
12         output_matrix[i, j] = np.max(region)
13
14 print(output_matrix)
15

```

Figura 6 – Código (5)
Fonte: Elaborado pelo autor (2024)

Output:

```
[[6. 4.]
 [3. 3.]]
```

Figura 7 – Código (6)

Fonte: Elaborado pelo autor (2024)

IMPLEMENTAÇÃO PRÁTICA DE UMA CNN SIMPLES

Vamos construir uma CNN simples utilizando o framework Keras para classificar imagens do dataset MNIST.

```
1 # Carregando o dataset MNIST e pré-processando
2 (train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
3 train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255
4 test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255
5
```

Figura 8 – Código (7)

Fonte: Elaborado pelo autor (2024)

Plotando as 25 primeiras imagens do dataset MNIST:1

```
1 # Definindo a classe dos dígitos de 0 a 9
2 class_names = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
3
4 # Plotando as primeiras 25 imagens do conjunto de treinamento
5 plt.figure(figsize=(10,10))
6 for i in range(25):
7     plt.subplot(5,5,i+1)
8     plt.xticks([])
9     plt.yticks([])
10    plt.grid(False)
11    plt.imshow(train_images[i], cmap=plt.cm.binary)
12    plt.xlabel(class_names[train_labels[i]])
13 plt.show()
14
```

Figura 9 – Código (8)

Fonte: Elaborado pelo autor (2024)

Output:

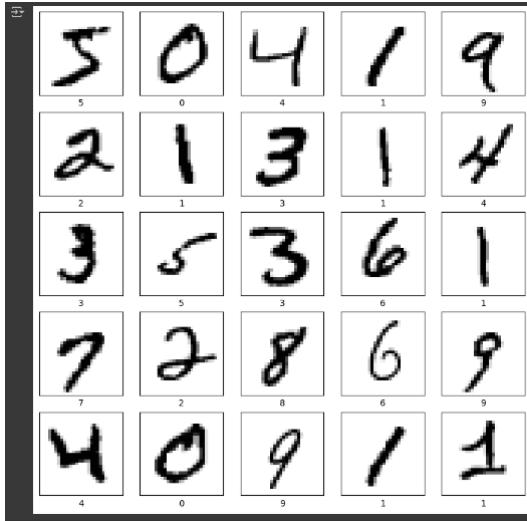


Figura 10 – Output
Fonte: Elaborado pelo autor (2024)

CAMADAS FULLY CONNECTED

As camadas fully connected (densamente conectadas) são usadas para a classificação final. Cada neurônio em uma camada fully connected está conectado a todos os neurônios da camada anterior.

```

1  # Construindo o modelo CNN
2  model = models.Sequential()
3  model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
4  model.add(layers.MaxPooling2D((2, 2)))
5  model.add(layers.Conv2D(64, (3, 3), activation='relu'))
6  model.add(layers.MaxPooling2D((2, 2)))
7  model.add(layers.Conv2D(64, (3, 3), activation='relu'))
8
9  # Adicionando camadas fully connected
10 model.add(layers.Flatten())
11 model.add(layers.Dense(64, activation='relu'))
12 model.add(layers.Dense(10, activation='softmax'))
13
14 model.summary()
15

```

Figura 11 – Código (9)
Fonte: Elaborado pelo autor (2024)

As camadas fully connected (densamente conectadas) são usadas para a classificação final. Cada neurônio em uma camada fully connected está conectado a todos os neurônios da camada anterior.

Output:

```

/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/'input_dim' at
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dense_1 (Dense)	(None, 10)	650

Total params: 91,122 (364.54 KB)
 Trainable params: 91,122 (364.54 KB)
 Non-trainable params: 0 (0.00 B)

Figura 12 – Código (10)
 Fonte: Elaborado pelo autor (2024)

Compilando o modelo

```

1 # Compilando o modelo
2 model.compile(optimizer='adam',
3               loss='sparse_categorical_crossentropy',
4               metrics=['accuracy'])
5
6 # Treinando o modelo
7 history = model.fit(train_images, train_labels, epochs=5,
8                     validation_data=(test_images, test_labels))
9

```

Figura 13 – Código (11)
 Fonte: Elaborado pelo autor (2024)

Output:

```

Epoch 1/5
1875/1875 — 17s 7ms/step - accuracy: 0.8943 - loss: 0.3413 - val_accuracy: 0.9851 - val_loss: 0.0470
Epoch 2/5
1875/1875 — 10s 3ms/step - accuracy: 0.9849 - loss: 0.0487 - val_accuracy: 0.9887 - val_loss: 0.0343
Epoch 3/5
1875/1875 — 11s 3ms/step - accuracy: 0.9891 - loss: 0.0317 - val_accuracy: 0.9904 - val_loss: 0.0300
Epoch 4/5
1875/1875 — 5s 3ms/step - accuracy: 0.9918 - loss: 0.0254 - val_accuracy: 0.9907 - val_loss: 0.0292
Epoch 5/5
1875/1875 — 10s 3ms/step - accuracy: 0.9931 - loss: 0.0199 - val_accuracy: 0.9911 - val_loss: 0.0271

```

Figura 14 – Código (12)
 Fonte: Elaborado pelo autor (2024)

Avaliando o modelo

```

1 # Avaliando o modelo
2 test_loss, test_acc = model.evaluate(test_images, test_labels)
3 print(f'Test accuracy: {test_acc}')
4

```

Figura 15 – Código (13)
 Fonte: Elaborado pelo autor (2024)

Output:

```
313/313 1s 2ms/step - accuracy: 0.9883 - loss: 0.0357
Test accuracy: 0.991100013256073
```

Figura 16 – Código – Output (14)
Fonte: Elaborado pelo autor (2024)

Acurácia e perda:

```
1 # Visualizando a acurácia e a perda durante o treinamento
2 plt.plot(history.history['accuracy'], label='accuracy')
3 plt.plot(history.history['val_accuracy'], label='val_accuracy')
4 plt.xlabel('Epoch')
5 plt.ylabel('Accuracy')
6 plt.ylim([0.5, 1])
7 plt.legend(loc='lower right')
8 plt.show()
9
```

Figura 17 – Código (15)
Fonte: Elaborado pelo autor (2024)

Output:

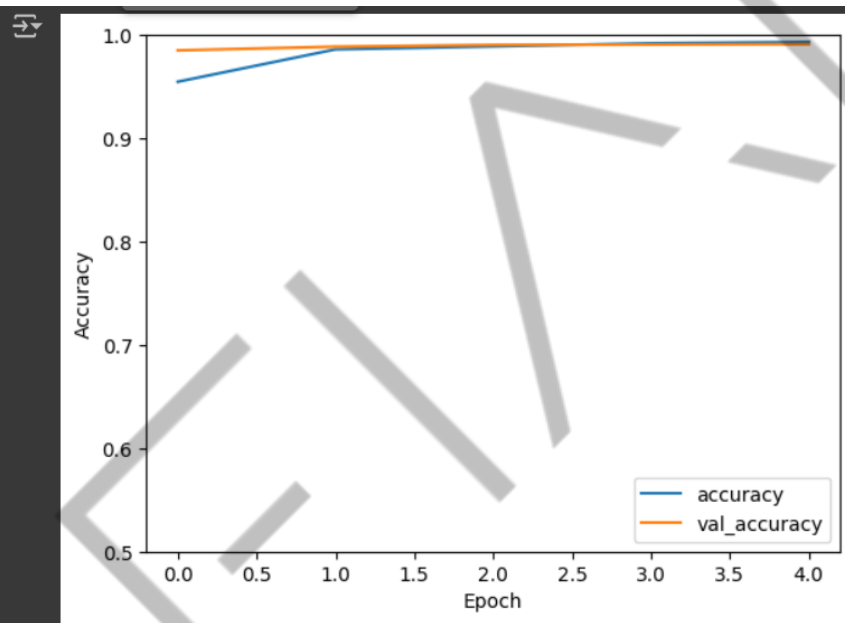


Figura 17 – Output
Fonte: Elaborado pelo autor (2024)

Gerando as previsões:

```

1 # Gerando as previsões para os dados de teste
2 predictions = model.predict(test_images)
3
4 # Plotando algumas imagens de teste com as previsões do modelo
5 plt.figure(figsize=(10,10))
6 for i in range(25):
7     plt.subplot(5,5,i+1)
8     plt.xticks([])
9     plt.yticks([])
10    plt.grid(False)
11    plt.imshow(test_images[i].reshape(28, 28), cmap=plt.cm.binary)
12    predicted_label = np.argmax(predictions[i])
13    true_label = test_labels[i]
14    if predicted_label == true_label:
15        color = 'blue'
16    else:
17        color = 'red'
18    plt.xlabel(f'Predicted: {predicted_label} \n True: {true_label}', color=color)
19 plt.show()
20

```

Figura 18 – Código (16)
Fonte: Elaborado pelo autor (2024)

Output:



Figura 19 – Output
Fonte: Elaborado pelo autor (2024)

SAIBA MAIS

As Redes Neurais Convolucionais (CNNs) têm sido uma das maiores inovações no campo da visão computacional, transformando a maneira como as máquinas interpretam e entendem imagens. Desde a sua introdução, as CNNs têm inspirado uma série de pesquisas e aplicações práticas, estabelecendo-se como uma ferramenta crucial na visão computacional e além.

As CNNs são conhecidas por sua capacidade de aprender automaticamente as características mais relevantes das imagens, permitindo uma precisão elevada em tarefas como reconhecimento de objetos, classificação de imagens e detecção de padrões.

A seguir, vamos fornecer uma visão abrangente e detalhada sobre a importância, evolução e aplicações das Redes Neurais Convolucionais, consolidando seu papel fundamental na área de visão computacional.

ORIGEM E NOME

O termo "Rede Neural Convolucional" foi cunhado para refletir o processo de convolução aplicado às imagens, onde filtros são utilizados para extrair características específicas. As CNNs foram desenvolvidas para imitar a forma como o cérebro humano processa informações visuais, e a palavra "convolucional" refere-se à operação matemática de convolução que é fundamental para seu funcionamento.

IMPACTO NA VISÃO COMPUTACIONAL

As CNNs causaram um impacto significativo na visão computacional devido à sua capacidade de processar e analisar imagens com uma precisão sem precedentes. Antes das CNNs, a análise de imagens dependia de técnicas manuais e características projetadas manualmente, o que limitava a eficácia e a aplicabilidade dos modelos. As CNNs, por outro lado, automatizam o processo de extração de características, permitindo a descoberta de padrões complexos em grandes volumes de dados visuais.

REDUÇÃO DE SOBRECARGA COMPUTACIONAL

Uma das maiores vantagens das CNNs é a redução significativa da sobrecarga computacional em comparação com métodos tradicionais de processamento de imagens. As CNNs utilizam filtros compartilhados e pooling, que reduzem a dimensionalidade dos dados sem perder informações críticas. Isso torna as CNNs altamente eficientes e escaláveis, permitindo seu uso em aplicações de tempo real.

EVOLUÇÃO CONSTANTE

Desde sua introdução, as CNNs passaram por várias iterações e aprimoramentos. As primeiras arquiteturas, como LeNet-5, abriram caminho para modelos mais sofisticados como AlexNet, VGG, ResNet e EfficientNet. Cada nova geração de CNNs trouxe melhorias substanciais em termos de profundidade, eficiência e precisão, impulsionando a pesquisa e o desenvolvimento contínuos na área de visão computacional.

COMUNIDADE E CONTRIBUIÇÕES

A pesquisa em CNNs é amplamente suportada por uma comunidade vibrante de pesquisadores(as), desenvolvedores(as) e entusiastas. A natureza open-source de muitas implementações de CNNs facilitou a colaboração e a inovação, permitindo que novos métodos e técnicas sejam rapidamente disseminados e adotados. Ferramentas populares como TensorFlow, PyTorch e Keras têm desempenhado um papel crucial na democratização das CNNs.

APLICAÇÕES DIVERSIFICADAS

As CNNs são aplicadas em uma ampla gama de domínios, incluindo reconhecimento facial, diagnóstico médico, veículos autônomos, robótica e análise de vídeo. Sua capacidade de detectar e classificar objetos em imagens com alta precisão tornou-as essenciais para muitas tecnologias modernas e emergentes.

INTEGRAÇÃO COM IOT E DISPOSITIVOS MÓVEIS

Graças à sua eficiência, as CNNs podem ser integradas em dispositivos de Internet das Coisas (IoT) e dispositivos móveis. Esta integração é especialmente útil para aplicações que exigem processamento local, como câmeras de segurança inteligentes e sistemas de direção autônoma. Modelos otimizados para execução em

dispositivos com recursos limitados permitem o uso de CNNs em uma variedade de contextos práticos.

TÉCNICAS DE DATA AUGMENTATION

As CNNs utilizam técnicas avançadas de aumento de dados (data augmentation) para melhorar a robustez dos modelos. Essas técnicas introduzem variações nos dados de treinamento, como rotações, zooms e inversões, ajudando a generalizar melhor os modelos para novos dados. Isso é crucial para melhorar o desempenho em cenários do mundo real, onde os dados podem variar amplamente.

INOVAÇÃO EM ARQUITETURAS

As versões mais recentes de CNNs introduziram conceitos inovadores como convoluções dilatadas, convoluções de profundidade separável e redes de atenção. Essas inovações permitem que as CNNs capturem contextos mais amplos e características mais detalhadas, melhorando a detecção de objetos pequenos e sobrepostos, que eram problemáticos nas primeiras versões.

PRÊMIOS E RECONHECIMENTO

As CNNs têm sido amplamente reconhecidas na comunidade de visão computacional, conquistando prêmios e destaque em competições e benchmarks. Arquiteturas como AlexNet e ResNet foram marcos na história das CNNs, estabelecendo novos padrões de desempenho e eficiência. A contínua pesquisa e desenvolvimento garantem que as CNNs permaneçam na vanguarda da tecnologia de visão computacional.

INTEGRAÇÃO COM PLATAFORMAS POPULARES

As CNNs se integram facilmente com várias plataformas de desenvolvimento populares, como TensorFlow, PyTorch e OpenCV. Isso torna o desenvolvimento e a implementação de modelos CNN acessíveis a uma ampla gama de desenvolvedores e pesquisadores. Ferramentas como o TensorFlow Hub e o PyTorch Hub permitem a reutilização de modelos pré-treinados, facilitando a aplicação de CNNs em novos projetos.

USO EM AMBIENTES RESTRINGIDOS

Devido à sua eficiência e rapidez, as CNNs são especialmente úteis em ambientes com restrições de hardware, como dispositivos móveis e sistemas embarcados. Modelos otimizados podem executar inferência em tempo real com recursos limitados, tornando as CNNs ideais para aplicações onde a capacidade de processamento é uma preocupação.

TRANSFER LEARNING E FINE-TUNING

As CNNs também suportam técnicas de transferência de aprendizado (transfer learning) e ajuste fino (fine-tuning). Isso permite que os desenvolvedores comecem com um modelo pré-treinado em um grande conjunto de dados e adaptem-no para suas necessidades específicas com um pequeno conjunto de dados rotulados, economizando tempo e recursos.

COMPETIÇÃO ENTRE OUTROS MODELOS DE VISÃO COMPUTACIONAL

As CNNs competem com outros modelos de visão computacional, como Redes Neurais Recorrentes (RNNs) e Transformers. Embora cada modelo tenha suas próprias vantagens, as CNNs se destacam pela sua eficácia na extração de características espaciais de imagens. A escolha entre esses modelos geralmente depende da natureza específica da tarefa e do equilíbrio necessário entre precisão e velocidade.

PESQUISA E DESENVOLVIMENTO CONTÍNUOS

A pesquisa em CNNs continua a evoluir, com novas técnicas e arquiteturas sendo desenvolvidas regularmente. Pesquisadores usam CNNs para experimentar novas abordagens em aprendizado profundo, como redes generativas adversárias (GANs) e aprendizado por reforço. Isso ajuda a manter as CNNs na vanguarda das inovações em visão computacional.

DESAFIOS E LIMITAÇÕES

Apesar de suas muitas vantagens, as CNNs ainda enfrentam desafios. A detecção de objetos muito pequenos ou muito próximos uns dos outros continua a ser uma área de melhoria. Além disso, o balanceamento entre a complexidade do modelo e a capacidade de execução em dispositivos de baixa potência é uma área de

pesquisa ativa. Técnicas como pruning e quantização são exploradas para otimizar o desempenho em hardware restrito.

CONTRIBUIÇÕES DA COMUNIDADE

A comunidade em torno das CNNs é vibrante e ativa. Contribuições incluem não apenas melhorias no código, mas também a criação de tutoriais, documentação e a adaptação das CNNs para diferentes casos de uso. A comunidade também compartilha conjuntos de dados e implementações que podem ser reutilizadas e adaptadas para diferentes projetos, promovendo a colaboração e o avanço contínuo.

FUTURO DAS CNNs

O futuro das CNNs parece promissor, com contínuas atualizações e melhorias. Com a crescente demanda por aplicações de visão computacional em áreas como veículos autônomos, robótica e monitoramento de saúde, o desenvolvimento das CNNs e suas capacidades adaptativas continuarão a ser um foco principal tanto na academia quanto na indústria. A evolução das CNNs promete trazer novas possibilidades e avanços na forma como interagimos com a tecnologia visual.

MERCADO, CASES E TENDÊNCIAS

As Redes Neurais Convolucionais (CNNs) têm sido aplicadas com sucesso em uma variedade de projetos reais em diferentes indústrias, proporcionando avanços significativos em diversas áreas. Abaixo estão alguns exemplos de estudos de caso e aplicações práticas de CNNs:

Diagnóstico médico:

- Detecção de câncer de mama: CNNs são utilizadas para analisar imagens de mamografias e identificar possíveis tumores, auxiliando radiologistas no diagnóstico precoce de câncer de mama.
- Diagnóstico de doenças dermatológicas: CNNs são empregadas na análise de imagens de lesões de pele para detectar e classificar diferentes tipos de doenças dermatológicas, como melanoma e psoríase.

Reconhecimento de objetos:

- Assistência à condução autônoma: CNNs são aplicadas em sistemas de visão computacional de veículos autônomos para detectar e reconhecer pedestres, veículos e obstáculos na estrada.
- Contagem e classificação de produtos em cadeias de suprimentos: CNNs são utilizadas para contar e classificar produtos em linhas de produção e cadeias de suprimentos, garantindo precisão e eficiência na logística.

Análise de imagens via satélite:

- Monitoramento ambiental: CNNs são empregadas na análise de imagens de satélite para monitorar mudanças ambientais, como desmatamento, poluição e mudanças climáticas.
- Agricultura de precisão: CNNs são utilizadas para analisar imagens de satélite e identificar padrões agrícolas, monitorar a saúde das plantações e prever safras, auxiliando os agricultores na tomada de decisões.

Reconhecimento de voz e tradução automática:

- Assistentes virtuais inteligentes: CNNs são aplicadas em sistemas de reconhecimento de voz, como Siri, Alexa e Google Assistant, para interpretar e responder a comandos de voz dos usuários.
- Tradução automática de texto: CNNs são utilizadas em sistemas de tradução automática, como o Google Translate, para converter texto de um idioma para outro com precisão e fluência.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, imergimos nas Redes Neurais Convolucionais (CNNs), explorando seus fundamentos, estruturas e aplicações práticas.

Começamos com uma introdução às Redes Neurais Convolucionais, discutindo sua importância e aplicabilidade na visão computacional. As CNNs são fundamentais para o reconhecimento de padrões em imagens, sendo amplamente utilizadas em tarefas como classificação de imagens, detecção de objetos e segmentação de imagens.

Exploramos a estrutura básica de uma CNN, que é composta principalmente de camadas convolucionais, camadas de pooling e camadas fully connected.

Passamos para a implementação prática de uma CNN usando bibliotecas populares como TensorFlow e Keras. Demonstramos como construir, treinar e avaliar um modelo CNN em um conjunto de dados de exemplo, como o MNIST (dígitos manuscritos) ou CIFAR-10 (imagens de objetos).

Exploramos diversas aplicações das CNNs em áreas como diagnóstico médico, reconhecimento facial, veículos autônomos e análise de vídeos. Discutimos avanços recentes em arquiteturas de CNNs, como ResNet, VGG e EfficientNet, que melhoraram significativamente a precisão e a eficiência dos modelos.

Discutimos alguns dos desafios e limitações das CNNs, incluindo a dificuldade em detectar objetos pequenos e próximos, a necessidade de grandes quantidades de dados rotulados para treinamento e a alta demanda computacional. Abordamos técnicas de otimização, como pruning e quantização, para mitigar essas limitações.

Finalizamos com uma visão sobre o futuro das CNNs, destacando áreas de pesquisa emergentes e inovações, como convoluções de profundidade separável e redes de atenção. Discutimos como essas inovações estão impulsionando novas possibilidades e aplicações na visão computacional.

REFERÊNCIAS

- CHOLLET, F. **Deep Learning with Python**. Nova York: Manning Publications, 2017.
- GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Massachusetts: O'Reilly Media, 2019.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge: MIT Press, 2016.
- HE, K.; ZHANG, X.; REN, S.; SUN, J. **Deep Residual Learning for Image Recognition**. In: CVPR, 2016.
- KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. **ImageNet Classification with Deep Convolutional Neural Networks**. In: NIPS, 2012.
- SZEGEDY, C.; LIU, W.; JIA, Y., et al. **Going Deeper with Convolutions**. In: CVPR, 2015.
- TAN, M.; LE, Q. V. **EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks**. In: ICML, 2019.

PALAVRAS-CHAVE

Redes Neurais Convolucionais, CNNs, Camadas Convolucionais, Pooling, Fully Connected, Deep Learning, Visão Computacional.

EMAP



POSTECH